

// Jivops

MariaDB

Guía completa — 20 lecciones

Fundamentos y motores · Rendimiento y diseño · Replicación, Galera y
MaxScale · Backup, seguridad y producción

Contenido

Módulo 1: Fundamentos y diferencias con MySQL

Lección 01 — Qué es MariaDB y su relación con MySQL

Lección 02 — Motores de almacenamiento de MariaDB

Lección 03 — Tablas con versionado de sistema

Lección 04 — Secuencias frente a AUTO_INCREMENT

Lección 05 — Thread pool y archivos de configuración

Módulo 2: Rendimiento y diseño

Lección 06 — InnoDB — buffer pool y redo log

Lección 07 — EXPLAIN y la sentencia ANALYZE

Lección 08 — Estadísticas independientes del motor e histogramas

Lección 09 — Concurrencia — deadlocks, NOWAIT y SKIP LOCKED

Lección 10 — Particionado de tablas

Módulo 3: Replicación, Galera y MaxScale

Lección 11 — Replicación asíncrona y GTID de MariaDB

Lección 12 — Galera Cluster — replicación síncrona multi-primaria

Lección 13 — Quórum, SST e IST en Galera

Lección 14 — Limitaciones de Galera y control de flujo

Lección 15 — MaxScale — proxy, separación de lecturas y failover

Módulo 4: Backup, seguridad y producción

Lección 16 — mariadb-backup — copias físicas en caliente

Lección 17 — Recuperación a un punto en el tiempo y pruebas de restauración

Lección 18 — Autenticación, unix_socket y roles

Lección 19 — TLS, cifrado en reposo y auditoría

Lección 20 — Proyecto final: MariaDB listo para producción

Módulo 1: Fundamentos y diferencias con MySQL

Lección 01 — Qué es MariaDB y su relación con MySQL

MariaDB nació en 2009 como una bifurcación (fork) de MySQL y al principio era un reemplazo directo, pero ha divergido: tiene motores, funciones y formatos propios (por ejemplo un GTID distinto), así que la compatibilidad ya no es total y hay que probar antes de migrar.

Han divergido en funciones, formatos y comportamiento (por ejemplo el GTID o algunos tipos como JSON); una característica que existe en uno puede faltar o comportarse distinto en el otro, y las pruebas detectan esas incompatibilidades antes de afectar a producción.

Lección 02 — Motores de almacenamiento de MariaDB

InnoDB es el motor por defecto para cargas transaccionales; MariaDB añade otros como Aria (alternativa a MyISAM resistente a fallos), MyRocks (basado en LSM, buena compresión y escrituras intensivas), ColumnStore (almacenamiento por columnas para analítica) y Spider (reparto de datos entre servidores).

Cada motor está optimizado para un patrón de uso distinto (analítica, escrituras intensivas) y puede mejorar el rendimiento o el espacio en esa tabla sin cambiar el resto; a cambio, pueden cambiar garantías como las transacciones.

Lección 03 — Tablas con versionado de sistema

Una tabla creada WITH SYSTEM VERSIONING conserva automáticamente las versiones anteriores de cada fila con su período de validez, y se pueden consultar estados pasados con FOR SYSTEM_TIME AS OF; ocupa más espacio y no sustituye a una copia de seguridad.

Cada actualización o borrado conserva la versión anterior, así que el historial solo aumenta; sin una política de retención o particionado, el espacio crece indefinidamente, aunque se puede limitar o particionar por tiempo.

Lección 04 — Secuencias frente a AUTO_INCREMENT

Un SEQUENCE genera números de forma independiente de las tablas (NEXT VALUE FOR o NEXTVAL), puede compartirse entre varias tablas y configurarse con incremento, límites y ciclo; AUTO_INCREMENT está ligado a una columna de una tabla concreta.

Un único contador genera números que no se repiten entre las distintas tablas, mientras que AUTO_INCREMENT es propio de cada tabla y cada una tendría su propia numeración; la secuencia desacopla el identificador de la tabla.

Lección 05 — Thread pool y archivos de configuración

Por defecto usa un hilo por conexión, pero incluye un thread pool (thread_handling=pool-of-threads) que atiende muchas conexiones con un número limitado de hilos y reduce el coste con muchas conexiones cortas; la configuración se lee de my.cnf y su directorio de configuración, en la sección [mariadb] o [mysqld].

Crear y destruir un hilo por conexión tiene un coste, y muchos hilos aumentan el consumo y el cambio de contexto; un conjunto limitado de hilos reutilizados atiende a muchas conexiones y reduce la sobrecarga.

Módulo 2: Rendimiento y diseño

Lección 06 — InnoDB — buffer pool y redo log

`innodb_buffer_pool_size` es la caché principal de datos e índices (en un servidor dedicado suele ser buena parte de la RAM, con una referencia habitual del 50 al 75 %), y el redo log determina cuánto trabajo puede acumular InnoDB antes de volcarlo a disco, lo que afecta a las escrituras sostenidas.

Cuando se llena, obliga a volcar páginas modificadas a disco con rapidez y las escrituras esperan; un log mayor absorbe picos y vuelca de forma más suave, aunque su tamaño debe equilibrarse con el tiempo de recuperación tras un fallo.

Lección 07 — EXPLAIN y la sentencia ANALYZE

EXPLAIN muestra el plan estimado sin ejecutar la consulta; ANALYZE SELECT (propio de MariaDB) la ejecuta y compara filas estimadas con filas reales, y ANALYZE FORMAT=JSON añade tiempos, lo que revela estimaciones erróneas; con sentencias que modifican datos hay que recordar que se ejecutan de verdad.

Una gran diferencia indica que el optimizador estimó mal y pudo elegir un plan inadecuado, algo que suele deberse a estadísticas ausentes o desactualizadas; así se corrige con estadísticas o índices en lugar de adivinar.

Lección 08 — Estadísticas independientes del motor e histogramas

MariaDB puede guardar estadísticas persistentes independientes del motor, incluidos histogramas por columna, que ayudan al optimizador a estimar mejor el filtrado sobre columnas sin índice o con datos muy sesgados; se recogen con ANALYZE TABLE ... PERSISTENT FOR y deben refrescarse cuando los datos cambian mucho.

Muestra cómo se distribuyen los valores y el optimizador puede estimar cuántas filas devolverá un filtro concreto; sin él supondría un reparto uniforme y se equivocaría con valores muy frecuentes o muy raros.

Lección 09 — Concurrency — deadlocks, NOWAIT y SKIP LOCKED

InnoDB aísla con REPEATABLE READ por defecto y puede producir deadlocks que la aplicación debe reintentar; SELECT ... FOR UPDATE admite NOWAIT y, en versiones recientes (SKIP LOCKED desde la 10.6), SKIP LOCKED, útil para que varios consumidores tomen tareas de una cola sin esperarse.

Cada worker coge filas libres sin esperar a que otro termine con las que ya bloqueó, así no se bloquean entre sí peleando por la misma fila y mejora el paralelismo de una cola implementada sobre una tabla.

Lección 10 — Particionado de tablas

Divide una tabla en partes (RANGE, LIST, HASH); el optimizador descarta las particiones que la consulta no necesita (partition pruning) si filtra por la clave de partición, y borrar datos antiguos es muy rápido eliminando una partición entera; la clave de partición debe formar parte de todas las claves únicas.

Elimina un segmento completo en bloque en lugar de borrar millones de filas una a una; un DELETE masivo genera muchos registros de log y bloqueos y deja huecos, mientras que quitar una partición es sobre todo una operación de metadatos.

Módulo 3: Replicación, Galera y MaxScale

Lección 11 — Replicación asíncrona y GTID de MariaDB

La replicación asíncrona usa el binary log; el GTID de MariaDB tiene formato dominio-servidor-secuencia y es distinto del de MySQL, así que no son intercambiables; con GTID, una réplica se reposiciona con `MASTER_USE_GTID` sin indicar archivo y posición.

Los formatos de GTID son distintos y cada servidor no entiende el identificador del otro; la replicación por posición puede funcionar, pero se pierde la ventaja de reposicionar con GTID, así que hay que validar la compatibilidad antes de mezclarlos.

Lección 12 — Galera Cluster — replicación síncrona multi-primaria

Replica de forma prácticamente síncrona entre nodos (wsrep): cualquier nodo acepta escrituras (multi-primario), el conjunto de cambios se envía a todos y pasa una certificación; si dos transacciones entran en conflicto, una gana y la otra falla en el commit.

Otra transacción en otro nodo pudo modificar las mismas filas y la certificación rechaza una de las dos; el conflicto se detecta al certificar el conjunto de cambios, no durante la ejecución local, y la aplicación debe reintentar la transacción fallida.

Lección 13 — Quórum, SST e IST en Galera

Galera necesita mayoría de nodos para seguir operando: por eso se usan 3 nodos o un árbitro (garbd), ya que con 2 una partición de red deja a ambos sin mayoría; un nodo que se une recibe los datos por SST (copia completa) o por IST (solo los cambios que le faltan) si están en la caché.

Podría aceptar escrituras que los demás nodos no ven y los datos divergirían; es la protección frente al split-brain, ya que al recuperar el contacto habría que reconciliar historias distintas.

Lección 14 — Limitaciones de Galera y control de flujo

Funciona sobre todo con InnoDB y conviene que todas las tablas tengan clave primaria; las transacciones muy grandes son costosas porque se replican como un bloque al confirmar, y el control de flujo hace que el nodo más lento limite la velocidad de escritura de todo el cluster.

Todos los nodos deben aplicar todas las escrituras, así que cada uno hace el mismo trabajo, y el control de flujo limita la velocidad al nodo más lento; más nodos mejoran disponibilidad y lecturas, pero no la escritura.

Lección 15 — MaxScale — proxy, separación de lecturas y failover

Es un proxy que entiende el protocolo de MariaDB: con el router `readwritesplit` envía las escrituras al primario y las lecturas a réplicas, vigila el estado de los servidores y puede automatizar el failover; hay que tener en cuenta el retraso de las réplicas para no leer datos desactualizados.

Si la lectura va a una réplica con retraso, esta aún no tiene el cambio recién escrito en el primario; puede limitarse el retraso aceptado o dirigir las lecturas críticas al primario, el compromiso de repartir lecturas entre réplicas asíncronas.

Módulo 4: Backup, seguridad y producción

Lección 16 — mariadb-backup — copias físicas en caliente

Hace copias físicas en caliente de InnoDB: se ejecuta `--backup`, después `--prepare` para aplicar el redo log y dejar la copia consistente, y por último `--copy-back` sobre un directorio de datos vacío con el servidor parado; admite copias incrementales y restaura mucho más rápido que un volcado lógico.

La copia se toma mientras se escribe y los archivos no son consistentes entre sí; `prepare` aplica el redo log para llevar los datos a un punto consistente, y sin ella el servidor podría no arrancar o tener datos incoherentes.

Lección 17 — Recuperación a un punto en el tiempo y pruebas de restauración

Se restaura la copia física, se anota la posición de binlog o GTID que registra el backup y se reproducen los binlogs con `mariadb-binlog` hasta el instante deseado; solo se sabe que el plan funciona si se restaura de verdad en otro entorno y se mide el tiempo de recuperación (RTO).

Pueden faltar archivos, estar corruptos o no ser consistentes sin que el comando de backup lo detecte; solo una restauración real lo comprueba, y descubrirlo durante un incidente es el peor momento.

Lección 18 — Autenticación, `unix_socket` y roles

Las cuentas se identifican por usuario y host; el plugin `unix_socket` permite autenticar por el usuario del sistema operativo (habitual para `root local`), los roles agrupan privilegios y se aplica el mínimo privilegio con `GRANT`; los plugins de validación de contraseñas ayudan a exigir contraseñas robustas.

Solo un usuario del sistema operativo autorizado en el propio servidor puede usar esa cuenta y no hay contraseña que robar; reduce la exposición de la cuenta más privilegiada a la red y deja el acceso ligado al control del sistema operativo.

Lección 19 — TLS, cifrado en reposo y auditoría

TLS cifra la conexión; el cifrado en reposo de tablas InnoDB y Aria, redo log y binlog se activa con un plugin de gestión de claves (`file_key_management` guarda las claves en un archivo del propio servidor, así que protege frente al robo de discos pero no frente a quien controle el host; es mejor un gestor externo), y el plugin `server_audit` registra conexiones y consultas.

Quien consiga acceso al servidor puede leer tanto los datos como el archivo de claves, así que solo protege frente al robo del disco o de una copia; un gestor de claves externo separa las claves de los datos.

Lección 20 — Proyecto final: MariaDB listo para producción

InnoDB con índices y estadísticas revisadas con `ANALYZE`, replicación o Galera de 3 nodos con MaxScale, `mariadb-backup` más binlogs para PITR con restauraciones ensayadas, usuarios con mínimo privilegio, TLS y cifrado en reposo con gestor de claves externo, y monitoreo del retraso, del control de flujo y de los conflictos.

Intenta mencionar al menos dos pilares: índices y estadísticas revisadas, Galera de 3 nodos o replicación con MaxScale, `mariadb-backup` más binlogs con restauraciones ensayadas, mínimo privilegio con TLS y cifrado, o monitoreo del retraso y los conflictos.